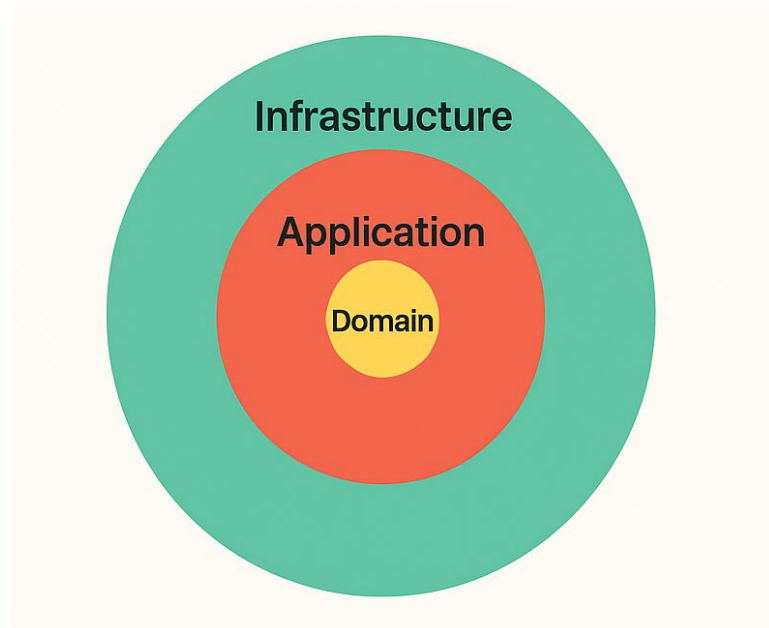


MÓDULO 2: ARQUITECTURA DE SOFTWARE

ESTRUCTURA DE CARPETAS EN UN PROYECTO CLEAN

APLICANDO CLEAN ARCHITECTURE CON TYPESCRIPT

Estructura opinionada de carpetas



Dos sabores: mínima y escalable

- **Mínima** para equipos pequeños/prototipos.
- **Escalable** para crecer con más adaptadores y módulos.

Estructura escalable

```
/src
/  domain
/    /entities
/    /value-objects
/    /services      # Domain services puros (si los necesitas)
/    /events        # Eventos de dominio (puros)
/    /errors        # Errores de dominio (p.ej. DomainError)
/  application
/    /use-cases     # Orquestación por caso de uso
/    /ports         # Interfaces de repos, buses, clocks, mailers...
/    /dto           # Tipos de entrada/salida (planos)
/    /errors        # Errores de aplicación (p.ej. ValidationError)
/  infrastructure
/    /persistence
/      /in-memory   # Dobles (fakes) para tests/arranque
/      /postgres    # Adaptadores reales (cuando toque)
/      /mappers     # Mapeo entidad <-> modelos de persistencia
/    /http
/      /controllers # Adaptadores de entrada (HTTP)
/      /routes      # Declaración de rutas
/      server.ts    # Bootstrap HTTP (Fastify/Express)
/      /messaging   # Kafka/Rabbit/etc (cuando toque)
/      /observability # Logger, metrics, tracing (adaptadores)
/      /config      # Carga de config (dotenv, etc.) – solo infra
/  composition
/    container.ts   # “Composición raíz” e inversión de dependencias
/  shared
/    result.ts      # Result/Either
/    utils          # Utilidades puras compartidas
main.ts            # Punto de entrada (levanta HTTP/CLI)
```

Estructura mínima

```
/src  
  /domain/{entities,value-objects,events}  
  /application/{use-cases,ports,dto}  
  /infrastructure/{persistence/http}  
  /composition/container.ts  
  /shared/result.ts  
main.ts
```

Convenciones de nombres y barreras

- **Entidades y VOs** en PascalCase (`Order`, `Price`, `SKU`, `Quantity`).
- **Puertos** con nombres del dominio + sufijo (`OrderRepository`, `PricingService`).
- **Adaptadores** con sufijo técnico (`InMemoryOrderRepository`, `PostgresOrderRepository`).
- **Casos de uso** en PascalCase con verbo (`CreateOrder`, `AddItemToOrder`).
- **Controladores HTTP** describen recurso + acción (`OrdersController.create`).
- Evita *barrels* (`index.ts`) que exporten a través de capas; límtalos a **subcarpetas del mismo nivel**.

Antipatrones

- Colocar **DTOs** en `infrastructure` (deben vivir en `application`).
- Importar `express/fastify/prisma` desde `application` o `domain`.
- Hacer `singleton global` del repositorio en `domain`.
- Usar `index.ts` que re-exporta capas cruzadas.
- Leer `process.env` fuera de `infrastructure/config`.



IA como Copiloto

- **Generar estructura inicial**

“Dado este árbol base de Clean Architecture {pega el árbol}, crea los archivos mínimos `.ts` por carpeta con comentarios `TODO` y ejemplos de imports con alias `TS`.”

- **Revisar barreras de import**

“Analiza este conjunto de imports y señala violaciones a la regla de dependencias (`domain` ↔ `application` ↔ `infrastructure`). Propón rutas válidas con alias.”

- **Sugerir nombres coherentes**

“Con este contexto de negocio {texto}, sugiere nombres de puertos, casos de uso y DTOs consistentes con el lenguaje ubicuo.”